

GUIDE SOUTENANCE — CONFIDENTIEL

CESIZen

Guide complet de soutenance Bloc 3
Ce que savoir, montrer et répondre au jury

- 1 Ce qui a été fait — résumé des livrables
- 2 Script de démonstration jury (6 min)
- 3 Concepts techniques à maîtriser
- 4 25 Questions & Réponses jury
- 5 Checklists pré-soutenance

Louis Beaujoin — CDA CESI Bloc 3 — Juillet 2026

Table des matières

1.	Ce qui a été fait — résumé des livrables	
1.1	Le code	
1.2	Docker & Infrastructure	
1.3	CI/CD & Tests	
1.4	Sécurité	
1.5	Maintenance & Ticketing	
2.	Script de démonstration jury	
2.1	Partie A — Versioning & CI/CD (3 min)	
2.2	Partie B — Ticketing (2 min)	
2.3	Partie C — Application live Docker (1 min)	
2.4	Scripts de secours si problème	
3.	Concepts techniques à maîtriser	
4.	25 Questions & Réponses jury	
4.1	Questions déploiement & Docker	
4.2	Questions CI/CD & Tests	
4.3	Questions sécurité OWASP	
4.4	Questions maintenance & SLA	
4.5	Questions Git & versioning	
5.	Checklists pré-soutenance	

1.1 Le code

Élément	Détail
Framework	Laravel 13 · PHP 8.3 · Tailwind CSS 4 · Vite 7
Application	Exercices de respiration · Pages info · Auth complète · Back-office admin · API REST
Sécurité code	SecurityHeadersMiddleware · throttle:5,1 · cast (int) XSS · bcrypt · @csrf · .env exclu
Tests	47 tests Pest · 118 assertions · SQLite in-memory · 2.51s

1.2 Docker & Infrastructure

Élément	Détail
Dockerfile	PHP 8.3 Apache + Node 20 + Composer · Build Vite intégré · .env exclus de l'image
docker-compose.yml	Service app (port 8080) + db MySQL 8 (healthcheck) · Volume persistant
entrypoint.sh	Copie .env.example → .env · key:generate · migrate --force · db:seed si DB_SEED=true
Données démo	admin@cesizen.fr / password · user@cesizen.fr / password · 3 exercices de respiration

1.3 CI/CD & Tests

Élément	Détail
GitHub Actions	2 jobs : tests (47 Pest) → docker build · déclenchement sur push main/develop
Critère go/no-go	Job docker bloqué si un seul test échoue
Dernière exécution	✅ success — 3 juillet 2026 — 1 min 44s
Dependabot	Composer hebdo · npm hebdo · GitHub Actions mensuel

1.4 Sécurité

Mécanisme	Vérification
4 en-têtes HTTP sécurité	Test automatisé + vérification live sur Docker (HTTP 200 + headers)

Throttle 429 après 5 tentatives	Test automatisé Pest + test HTTP live sur /connexion
Protection XSS	Test automatisé + (int) cast dans BreathingController
OWASP Top 10	7 risques couverts documentés dans le dossier technique

1.5 Maintenance & Ticketing

Élément	Détail
GitHub Issues	Labels (bloquant-critique / majeur / mineur) · Templates bug + feature
GitHub Projects	Board Kanban 4 colonnes (Backlog → En cours → En revue → Terminé)
SLA formalisé	Bloquant critique <1h prise en compte / <3h correction
Pull Request template	Checklist tests + sécurité + revue obligatoire

Durée totale : 6 minutes (dans les 20 minutes de soutenance)

Partie A : Versioning + CI/CD (3 min) · Partie B : Ticketing (2 min) · Partie C : App live (1 min)

2.1 Partie A — Versioning & CI/CD (3 min)

1

Montrer l'historique Git sur GitHub (30 sec)

1. Ouvrir github.com/louisbeaujoin/cesizen
2. Cliquer sur **le nombre de commits** affiché
3. Montrer les commits avec leurs types : `feat:` `fix:` `docs:` `test:`

"Chaque commit correspond à une étape précise. Le message conventionnel explique ce qui a été fait et pourquoi — pas juste 'mise à jour'."

2

Montrer le pipeline CI — dernier run (1 min 30)

1. Onglet **Actions**
2. Cliquer sur le dernier workflow `✓ success`
3. Montrer **Job "Tests"** → 47 tests passent
4. Montrer **Job "Docker Build"** → image construite
5. Montrer le lien `needs: tests` qui bloque le job Docker

"À chaque push, les 47 tests s'exécutent automatiquement. Si un seul échoue, l'image Docker n'est pas construite. C'est le filet de sécurité qui empêche du code cassé d'atteindre le serveur."

3

Montrer les branches (30 sec)

1. Cliquer sur le sélecteur de branches
2. Montrer `main` et `develop`
3. Montrer les Pull Requests fusionnées

"Le code ne va jamais directement en production. Develop → PR → main. Chaque merge est traçable."

2.2 Partie B — Ticketing (2 min)

4

Montrer le board GitHub Projects (1 min)

1. Onglet **Projects** → ouvrir "CESIZen — Suivi du projet"
2. Montrer les 4 colonnes et les tickets positionnés
3. Ouvrir un ticket `bug` — montrer le template rempli

"Le board reflète l'état réel du projet. Chaque ticket a une criticité, un SLA associé, et un lien vers le commit qui le corrige."

5

Montrer Dependabot (1 min)

1. Onglet **Security** ou **Pull Requests**
2. Montrer les PRs Dependabot fermées (ou ouvertes)

"Dependabot vérifie chaque semaine si mes dépendances ont des mises à jour ou des vulnérabilités. Il crée automatiquement des PRs que je peux merger après avoir vérifié que les tests passent toujours."

2.3 Partie C — Application live Docker (1 min)

6

Montrer l'application sur `http://localhost:8080`

1. Ouvrir le navigateur → **`http://localhost:8080`**
2. Montrer la page d'accueil → exercices de respiration
3. Se connecter : `admin@cesizen.fr` / `password`
4. Montrer le back-office admin
5. Ouvrir les DevTools → Network → Montrer les en-têtes de sécurité

"L'application tourne dans Docker sur ma machine. Vous voyez les en-têtes de sécurité : X-Frame-Options, X-Content-Type-Options... Aucune dépendance à un serveur externe."

2.4 Scripts de secours si problème

Docker ne démarre pas

```
docker compose down -v  
docker compose up -d --build  
docker compose exec app php artisan db:seed --force
```

Pas de données de démo

```
docker compose exec app php artisan db:seed --force
```

Conteneur planté

```
docker compose logs app --tail 30    # Voir l'erreur  
docker compose restart app           # Relancer
```

CI/CD

CI = Continuous Integration : chaque push déclenche une suite de tests automatiques pour détecter les régressions immédiatement.

CD = Continuous Delivery : si les tests passent, l'artefact (image Docker) est construit et prêt à être déployé.

Dans CESIZen : `git push` → GitHub Actions → 47 tests Pest → Build Docker . Le CD s'arrête si le CI échoue (critère go/no-go).

Docker & Conteneurs

Un conteneur isole l'application dans un environnement reproductible. **L'image** est le modèle (Dockerfile), **le conteneur** est l'instance en cours d'exécution.

`docker compose up` démarre plusieurs conteneurs à la fois et gère les dépendances (app attend db). Le healthcheck MySQL garantit que l'app ne démarre pas avant que la BDD soit prête.

OWASP Top 10

Standard mondial de sécurité web. Les 7 risques couverts dans CESIZen :

- **A01 Contrôle d'accès** : middleware `auth` , rôles admin/user
- **A02 Cryptographie** : bcrypt 12 rounds, HTTPS en prod
- **A03 Injection** : Eloquent ORM (requêtes préparées), validation Laravel
- **A05 Mauvaise config** : APP_DEBUG=false, .env exclu de git
- **A07 Auth** : throttle 5 req/min, sessions sécurisées
- **XSS** : Blade `{{ }}` , cast `(int)` sur paramètres JS
- **CSRF** : token `@csrf` sur tous les formulaires

Semantic Versioning (SemVer)

Convention : **MAJEUR.MINEUR.PATCH**

- `v1.0.0` → première version stable
- `v1.1.0` → nouvelle fonctionnalité rétrocompatible
- `v1.0.1` → correctif de bug
- `v2.0.0` → changement cassant l'API

Git Flow simplifié


```
main ← code stable (PRs seulement) ← tagged v1.0.0
└─ develop ← intégration continue
    └─ feature/* ← fonctionnalité isolée
```

SLA (Service Level Agreement)

Engagement contractuel sur les délais de prise en charge et de résolution par niveau de criticité :

- **Bloquant critique** (service indisponible) : prise en compte <1h, correction <3h
- **Bloquant majeur** (fonctionnalité principale dégradée) : <4h / <24h
- **Mineur** (gêne sans impact métier) : <48h / prochain sprint

RGPD

Règlement Général sur la Protection des Données (EU 2018). Points clés :

- **Minimisation** : ne collecter que le nécessaire (pas de tracking dans CESIZen)
- **Conservation limitée** : comptes inactifs >3 ans supprimés
- **Droits utilisateurs** : accès, rectification, effacement
- **Notification violation** : CNIL sous 72h, utilisateurs sous 72h

Throttle & Brute Force

Le middleware `throttle:5,1` limite à 5 requêtes par minute sur les routes de connexion. Au-delà : `HTTP 429 Too Many Requests`. Empêche les attaques par dictionnaire sur les mots de passe.

4.1 Déploiement & Docker

Q : Pourquoi Docker et pas WAMP pour l'environnement de test ?

Docker garantit un environnement identique pour tout le monde, reproductible en une commande. WAMP crée le syndrome "ça marche chez moi mais pas chez vous". Docker élimine ce risque : même version de PHP, MySQL, Node — partout.

Q : Expliquez le fichier `entrypoint.sh`

C'est le script qui démarre automatiquement au lancement du conteneur. Il copie `.env.example` en `.env` (les secrets réels ne sont jamais dans l'image), génère une clé Laravel, attend que MySQL soit prêt (boucle until), puis exécute les migrations. Cela rend le démarrage entièrement automatique, sans intervention manuelle.

Q : Pourquoi `.env` n'est pas dans l'image Docker ?

Le `.env` contient les mots de passe de base de données, les clés secrètes. Si l'image était publiée sur Docker Hub, ces secrets seraient exposés. On utilise `.env.example` (sans valeurs sensibles) comme modèle, et on passe les vraies valeurs via des variables d'environnement au runtime.

Q : Comment déployez-vous en production ?

En production (théorique) : VPS Linux, clone du repo, `.env` configuré avec les vraies valeurs, `php artisan migrate --force` et `php artisan config:cache`, Nginx + PHP-FPM, SSL Let's Encrypt. Dans ce projet, la prod est décrite théoriquement — l'environnement Docker TEST est celui réellement opérationnel.

4.2 CI/CD & Tests

Q : Pourquoi GitHub Actions plutôt que Jenkins ?

GitHub Actions est natif à GitHub : zéro configuration externe, tout est au même endroit (code + CI + tickets + PR). Jenkins nécessite un serveur dédié à maintenir. Pour un projet solo ou une petite équipe, Actions est le bon choix.

Q : Expliquez le critère go/no-go dans votre pipeline

Le job Docker a `needs: tests`. Si un seul des 47 tests échoue, GitHub Actions annule le job Docker automatiquement. C'est le go/no-go : les tests valident le code avant qu'il soit packagé. Code cassé = pas de build.

Q : Pourquoi 47 tests et pas plus ?

47 tests couvrent les chemins critiques : authentification, CRUD des modèles, accès admin, routes publiques, et maintenant 3 tests de sécurité (en-têtes, throttle, XSS). L'objectif n'est pas la quantité mais la pertinence : chaque test vérifie un comportement qui a une valeur métier ou sécuritaire.

Q : Qu'est-ce que RefreshDatabase dans vos tests ?

Le trait `RefreshDatabase` exécute toutes les migrations dans une base SQLite en mémoire avant chaque test, puis les annule après. Chaque test commence avec une base vide et propre. Cela garantit l'isolation des tests et évite les effets de bord entre eux.

4.3 Sécurité OWASP

Q : Expliquez l'injection XSS que vous avez corrigée

La page de respiration reçoit des paramètres d'URL (`?inspiration=4`) qui sont injectés dans du JavaScript. Sans protection, un attaquant passe `?inspiration=5;alert(1)//` et exécute du JS arbitraire. Le correctif : cast `(int)` dans le contrôleur — `(int)"5;alert(1)//"` retourne `5`. L'injection est neutralisée.

Q : À quoi servent les en-têtes X-Frame-Options et X-Content-Type-Options ?

X-Frame-Options: SAMEORIGIN empêche l'application d'être chargée dans un iframe sur un autre site (protection contre le clickjacking). **X-Content-Type-Options: nosniff** empêche le navigateur de deviner le type MIME d'une réponse — il respecte ce que le serveur déclare, évitant l'exécution de fichiers malveillants déguisés.

Q : Comment fonctionne le throttle contre le brute force ?

Laravel compte les requêtes par IP sur la route `/connexion`. Après 5 POST en moins d'une minute, il retourne `HTTP 429 Too Many Requests` automatiquement. L'attaquant ne peut pas tester 1000 mots de passe par minute — il est bloqué au bout de 5. Vérifié en test automatisé ET en test live sur l'app Docker.

Q : Pourquoi bcrypt plutôt que MD5 ?

MD5 est rapide → une carte graphique peut tester des milliards de hash par seconde. Bcrypt est intentionnellement lent (12 rounds dans CESIZen = ~300ms par hash). Même si la base de données est volée, casser les mots de passe prend des années. MD5 est brisé en secondes avec une rainbow table.

4.4 Maintenance & SLA

Q : Pourquoi GitHub Issues plutôt que Jira ou Trello ?

GitHub centralise tout dans le même outil : code + CI + tickets. La traçabilité est directe : un commit peut référencer un ticket (`Fixes #13`) et le fermer automatiquement. Jira nécessite un abonnement et une intégration séparée. Pour ce projet, GitHub Issues est suffisant et plus cohérent.

Q : Que se passe-t-il concrètement quand un bug critique est signalé ?

1. Ticket créé via le template (étapes de reproduction, criticité cochée). 2. Je le prends en compte sous 1h, je le labelle `bloquant-critique` . 3. Je crée une branche `fix/` depuis main, je corrige, je lance les tests. 4. PR → review → merge sur main <3h après signalement. 5. Tag SemVer patch (v1.0.1). Le ticket est fermé automatiquement par le merge.

Q : Comment Dependabot vous aide dans la maintenance ?

Dependabot surveille automatiquement les versions de mes dépendances (Composer, npm, GitHub Actions). Chaque semaine, il crée des PRs pour les mises à jour disponibles. Je vérifie le CHANGELOG de la dépendance, je m'assure que les 47 tests passent encore, et je merge. C'est la veille technologique automatisée.

4.5 Git & Versioning

Q : Expliquez votre Git Flow

`main` = code stable, jamais de push direct. `develop` = intégration des features. `feature/*` = fonctionnalité isolée créée depuis develop. Flux : feature → PR → develop → PR → main → tag SemVer. Chaque étape est traçable.

Q : C'est quoi le Semantic Versioning ?

Convention MAJEUR.MINEUR.PATCH. `v1.0.0` = première stable. `v1.1.0` = nouvelle fonctionnalité sans casser l'existant. `v1.0.1` = correctif de bug. `v2.0.0` = changement qui casse la compatibilité.

N'importe qui peut comprendre l'ampleur d'un changement rien qu'en regardant le numéro.

Q : Qu'est-ce que les Conventional Commits ?

Convention de message de commit : `type(scope): description` . Types : `feat` (nouvelle fonctionnalité), `fix` (correction), `docs` (documentation), `test` (tests), `chore` (maintenance). Permet de générer automatiquement un CHANGELOG et de comprendre l'historique sans lire le code.

Q : Que feriez-vous si un bug critique est découvert en production après un déploiement ?

1. `git revert <hash>` sur main pour annuler le commit fautif (préserve l'historique). 2. Si la BDD est impactée : `php artisan migrate:rollback` puis restauration mysqldump. 3. Push → CI → redéploiement automatique du code précédent. 4. Ticket `bloquant-critique` ouvert. 5. Correction en branche `fix/` puis re-merge.

Q : Pourquoi ne jamais commiter .env ?

Le `.env` contient des secrets réels : mots de passe BDD, clé `APP_KEY`, clés API. Une fois dans git, même supprimé plus tard, il reste dans l'historique et peut être retrouvé. GitHub scanne les repos publics et alerte. La règle : `.env` dans `.gitignore` dès le premier commit. On commit `.env.example` avec des valeurs fictives comme modèle.

Q : C'est quoi le Makefile et pourquoi l'utiliser ?

Un Makefile définit des raccourcis pour des commandes longues. `make up` remplace `docker compose up -d --build` . C'est de la documentation exécutable : n'importe qui peut lancer le projet sans connaître Docker en détail. Sur Windows sans make, on utilise directement les commandes `docker compose`.

La veille de la soutenance

- ☐ Docker Desktop est lancé et les conteneurs démarrent : `docker compose up -d --build`
- ☐ `http://localhost:8080` répond HTTP 200
- ☐ Les données de démo sont seedées : connexion `admin@cesizen.fr` / password fonctionne
- ☐ GitHub Actions : dernier run  success sur develop
- ☐ Le board GitHub Projects est à jour
- ☐ Les 3 documents PDF sont ouverts et prêts : Dossier, Présentation, Guide
- ☐ Navigateur ouvert sur `github.com/louisbeaujoin/cesizen`

Le matin de la soutenance

- ☐ Relancer `docker compose up -d` (pas besoin de rebuild)
- ☐ Vérifier `http://localhost:8080`
- ☐ Ouvrir la présentation PDF (15 slides)
- ☐ Ouvrir GitHub dans le navigateur
- ☐ Ouvrir les DevTools (F12) → onglet Network prêt pour montrer les en-têtes
- ☐ Réviser les Q&R du chapitre 4 (30 min)

Si quelque chose ne marche pas pendant la démo

- ☐ Garder son calme — le jury évalue aussi la gestion du stress
- ☐ Dire : "Je vais relancer le conteneur, c'est une procédure standard."

☐ Lancer : `docker compose restart app`

☐ Si ça ne marche pas : `docker compose down -v && docker compose up -d --build`

☐ En dernier recours : montrer les screenshots dans le dossier document/

☐ Ne pas paniquer — le jury sait que les démos en direct ont des aléas

Phrase de conclusion pour la soutenance

"CESIZen démontre qu'une application web peut être déployée, testée, sécurisée et maintenue avec une chaîne d'outils cohérente — Git, Docker, GitHub Actions, Pest — et que chaque décision technique est documentée, justifiable et réversible."